On the Naming of Packs

Document #:	P2994R0
Date:	2023-10-08
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Introduction

C++11 introduced variadic templates, which dramatically changed the way we write all sorts of code. Even if the only thing you can do with a parameter pack was expand it. C++17 extended this support to fold-expressions (and there's ongoing work to extend fold-expressions in a few directions [P2355R1] or [circle-recurrence]), but we still don't have any operations that apply to the pack itself - as opposed to consuming the whole thing.

It's tricky to add support for this, because we have to be careful that the syntax for applying an operation on the pack itself is distinct from the syntax from applying that operation on an element, otherwise we could end up with ambiguities. For example, with indexing, we cannot use `pack[0]` to be the first element in the pack, because something like `f(pack + pack[0]...)` is already a valid expression that cannot mean adding the first element of the pack to every element in the pack.

As such, all the ongoing efforts to add more pack functionality choose a distinct syntax. Consider the following table, where `elem` denotes a single variable and `pack` denotes a function parameter pack:

	Single Element	Pack
Indexing [P2662R2]	`elem[0]`	`pack...[0]`

Expansion Statement[1] **[P1306R1]**	`template for (auto x : elem)`	One of:
		`template for (auto x : {pack...})`
		`template for ... (auto x : pack)`
		`for ... (auto x : pack)`

Reflection [P1240R2]	`^elem`	🤷
Splice [P1240R2]	`[: elem :]`	`... [: pack :] ...`

As you can see, the syntaxes on the right are distinct from the syntaxes on the left - which is important for those syntaxes to be viable.

What is unfortunate, in my opinion, is that the syntaxes on the right are also all distinct from each other. We don't have orthogonality here - knowing how to perform an operation on an element doesn't necessarily inform you of the syntax to perform that operation on a pack. There's an extra `...` that you have to type, but where does it go? It depends.

Additionally, while the indexing syntax here, `pack...[0]`, works just fine for indexing, it can't be generalized to the other operations:

— `template for (auto x : pack...)` looks like a pack expansion, which would then suggest that `template for (auto x : a, b)` is valid too. But this leads us to conflicting meanings: are we iterating over each element listed (as we want for a pack) or are we iterating over the sub-elements of the one element listed (as we want for a tuple). These conflict over what `template for (auto x : a)` mean.

— `^pack...` is a pack expansion of a reflection, as in `vector{^pack...}` producing a `vector<meta::info>` with the reflections over all the elements of `pack`.`

While having all the functionality available to us in the various proposals would be great, I think we can do better.

Note that lambda capture isn't included in the above table, since `[pack...]` is basically a regular pack expansion and `[...pack1=pack2]` is introducing a new pack, which is a slightly different operation than what I'm talking about here.

§ 2 Proposal

I would like to suggest that rather than coming up with a bespoke syntax for every pack operation we need to do - that we instead come up with a bespoke syntax for *naming a pack*[2] and use *that* syntax for each operation. This gives us orthogonality and consistency.

Thankfully, we don't even really need to come up with what the syntax should be for naming a pack - we already kind of have it: `...pack`. This is basically the way that packs are introduced in various templates and lambda init-capture already.

Applying that syntax to each of the facilities presented in the previous section:

	Single Element	Pack (Previous)	Pack (Proposed)
Indexing	`elem[0]`	`pack...[0]`	`...pack[0]`

Expansion Statement	`template for (auto x : elem)`	One of:
		`template for (auto x : {pack...})`
		`template for ... (auto x : pack)`
		`for ... (auto x : pack)`
			`template for (auto x : ...pack)`

Reflection	`^elem`	🤷	`^...pack`
Splice	`[: elem :]`	`... [: pack :] ...`	`[: ...pack :] ...`

Here, all the syntaxes in the last column are the same as the syntaxes in the first column, except using `...pack` instead of `elem`. These syntaxes are all distinct and unambiguous.

It is pretty likely that many people will prefer at least one syntax in the second column to its corresponding suggestion in the third column (I certainly do). But, on the whole, I think the consistency and orthogonality outweigh these small preferences.

Incidentally, this syntax also avoids the potential ambiguity mentioned in [P2662R2] for its syntax proposal (that having a parameter of `T...[1]` is valid syntax today, which with this proposal for indexing into the pack would instead be spelled `...T[1]` which is not valid today), although given that two compilers don't even support that syntax today makes this a very minor, footnote-level advantage.

Note that this syntax does lead to one obvious question:

```
template <typename... Ts>
void foo(Ts... pack) {
    // if this is how we get the first element of the pack
    auto first = ...pack[0];

    // ... and this is how we iterate over it
    template for (auto elem : ... pack) { }

    // ... then what does this mean??
    auto wat = ...pack;
}
```

Maybe there's something interesting that `...pack` might mean, like some language-tuple thing. I can't immediately come up with a use-case though. So I think a decent enough answer to that question is: it means nothing. It's just not allowed. In the same way that when invoking a non-static member function, `x.f(y)` is a valid expression but `x.f` by itself is not. If in the future, somebody finds a use-case, we can also make it work later. The various pack operations (indexing, expansion, reflection, splice) are just bespoke grammar rules that happen to share this common `...pack` syntax - despite `...pack` itself having no meaning.

2.1 Wording

None of these proposals exist in the working draft today, so there's no wording to offer against the working draft. However, I can provide a diff against the [P2662R2] wording, which is pretty small (the actual feature is unchanged, only the syntax for it, which is to say only the grammar).

Change the grammar in [expr.prim.pack.index] (the rest of the description is fine):

```
pack-index-expression:
-   identifer ... [ constant-expression ]
+   ... identifer [ constant-expression ]
```

Change the grammar in [dcl.type.pack.indexing] (likewise, the rest of the description remains unchanged):

```
pack-index-specifier:
-   typedef-name ... [ constant-expression ]
+   ... typedef-name [ constant-expression ]
```

Change the example in [dcl.type.decltype] to use `...pack[0]` instead of `pack...[0]`.

Change the added sentence in [temp.type]:

> For a template parameter pack `T`, ~~`T...[constant-expression]`~~ `...T[constant-expression]` denotes a unique dependent type.

Remove the added annex C entry in [decl.array], since there's no longer any ambiguity.

3 References

[circle-recurrence] Sean Baxter. 2023-05-02. `[recurrence]`.
https://github.com/seanbaxter/circle/blob/master/new-circle/README.md#recurrence

[P1240R2] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. 2022-01-14. Scalable Reflection.
https://wg21.link/p1240r2

[P1306R1] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2019-01-21. Expansion statements.
https://wg21.link/p1306r1

[P2355R1] S. Davis Herring. 2023-02-13. Postfix fold expressions.
https://wg21.link/p2355r1

[P2662R2] Corentin Jabot, Pablo Halpern. 2023-07-15. Pack Indexing.
https://wg21.link/p2662r2

---

1. For expansion statements, even though we've agreed on the `template for` syntax, there does not appear to be a published document that uses that syntax. Also, the last revision doesn't have support for expanding over a pack due to the lack of syntax - the three options presented here are various ideas that have come up in various conversations with people.↩

2. What I mean by naming a pack here is basically using the pack as an operand in some function directly, as opposed to being part of a pack expansion. I think of this as naming the pack itself. Maybe referring to this problem as having a pack operand might be more to the point.↩